

AI USED: Chat GPT

AI percentage: 20%

```
class FeedForwardNet(nn.Module):
    def __init__(self, input_dim, hidden_sizes=(64, 32), n_classes=7):
        super().__init__()

        layers = []

        previous_dim = input_dim

        for hidden_dim in hidden_sizes:
            layers.append(nn.Linear(previous_dim, hidden_dim))
            layers.append(nn.ReLU())

        Show more ▾
```



Thanks. I can see why Experiment C is missing.

The issue is that your `FeedForwardNet` ignores the `dropout` argument. To make Experiment C work, you need to **modify the class** so it inserts a `Dropout` layer after each hidden layer when `dropout > 0`.

Replace **just your** `FeedForwardNet` **class** with this version:

```
Python  
class FeedForwardNet(nn.Module):
    def __init__(self, input_dim, hidden_sizes=(64, 32), n_classes=7, dropout=0.0):
        super().__init__()

        layers = []

        previous_dim = input_dim
```



+ Ask anything



can u put everything together

Yes. Put everything in **one cell** (or split into a few cells if your lecturer prefers). This includes initialization, training, plotting loss, plotting the fit, and the **tanh removed comparison**.

```
<> Python   Run

import numpy as np
import matplotlib.pyplot as plt

# =====
# Part 1.2: Hidden layers and activations
# 1 input -> 8 hidden neurons -> 1 output
# =====

# Initialise parameters
np.random.seed(42)

W1 = np.random.randn(1, 8) * 0.5
b1 = np.zeros(8)

W2 = np.random.randn(8, 1) * 0.5
b2 = np.zeros(1)

lr = 0.01
losses = []

# Training Loop
for step in range(3000):
```

+ Ask anything  

can u put everything together

Yes. Put everything in **one cell** (or split into a few cells if your lecturer prefers). This includes initialization, training, plotting loss, plotting the fit, and the **tanh removed comparison**.

```
<> Python   Run

import numpy as np
import matplotlib.pyplot as plt

# =====
# Part 1.2: Hidden layers and activations
# 1 input -> 8 hidden neurons -> 1 output
# =====

# Initialise parameters
np.random.seed(42)

W1 = np.random.randn(1, 8) * 0.5
b1 = np.zeros(8)

W2 = np.random.randn(8, 1) * 0.5
b2 = np.zeros(1)

lr = 0.01
losses = []

# Training Loop
for step in range(3000):
```

+ Ask anything  

```
git show --stat HEAD
commit 3824c5015fddf0d9c9636387062630441db70adb (HEAD ->
main)
Author: Marfo Mavis <mavis.marfo@ashesi.edu.gh>
Date: Mon Jul 27 17:40:17 2026 +0000
```

Complete Part 1.1

lab_3.ipynb | 163

+++++

Show more ▾



Good. This output tells us something important:

Your current HEAD is:

3824c50 Complete Part 1.1



That means you are **already back before the "Complete Student Reasoning section" commit** (ffba075). So you have not lost Part 0 or Part 1.1.

Your history is currently probably like this:

3824c50 Complete Part 1.1 ← you are here
f3553db Complete Part 0 setup
d87ea6f Initial commit



can u put everything together

Yes. Put everything in **one cell** (or split into a few cells if your lecturer prefers). This includes initialization, training, plotting loss, plotting the fit, and the **tanh removed comparison**.

<> Python



```
import numpy as np
import matplotlib.pyplot as plt
```

```
# =====
# Part 1.2: Hidden Layers and activations
# 1 input -> 8 hidden neurons -> 1 output
# =====
```

```
# Initialise parameters
np.random.seed(42)
```

```
W1 = np.random.randn(1, 8) * 0.5
b1 = np.zeros(8)
```

```
W2 = np.random.randn(8, 1) * 0.5
b2 = np.zeros(1)
```

```
lr = 0.01
losses = []
```

```
# Training Loop
for step in range(3000):
```



+ Ask anything



Thanks. I can see why Experiment C is missing.

The issue is that your `FeedForwardNet` ignores the `dropout` argument. To make Experiment C work, you need to **modify the class** so it inserts a `Dropout` layer after each hidden layer when `dropout > 0`.

Replace **just your** `FeedForwardNet` **class** with this version:

```
Python Run

class FeedForwardNet(nn.Module):
    def __init__(self, input_dim, hidden_sizes=(64, 32), n_classes=7, dropout=0.0):
        super().__init__()

        layers = []

        previous_dim = input_dim

        for hidden_dim in hidden_sizes:
            layers.append(nn.Linear(previous_dim, hidden_dim))
            layers.append(nn.ReLU())

            if dropout > 0:
                layers.append(nn.Dropout(dropout))

            previous_dim = hidden_dim
```

```
ValueError                                Traceback (most recent call last)
Cell In[7], line 19
     15
     16 # Training loop
     17 for step in range(3000):
     18
--> 19     z1 = x @ W1 + b1
     20     h = np.tanh(z1)
     21
     22     y_hat = h @ W2 + b2

Show more
```

This error means the **shape of `x` does not match the shape of `W1`**.

The important part is:

```
size 1 is different from 100
```

Your code expects:

```
Python Run

x @ W1
```

Thanks. I can see why Experiment C is missing.

The issue is that your `FeedForwardNet` ignores the `dropout` argument. To make Experiment C work, you need to **modify the class** so it inserts a `Dropout` layer after each hidden layer when `dropout > 0`.

Replace **just your** `FeedForwardNet` **class** with this version:

```
Python
class FeedForwardNet(nn.Module):
    def __init__(self, input_dim, hidden_sizes=(64, 32), n_classes=7, dropout=0.0):
        super().__init__()

        layers = []

        previous_dim = input_dim

        for hidden_dim in hidden_sizes:
            layers.append(nn.Linear(previous_dim, hidden_dim))
            layers.append(nn.ReLU())

            if dropout > 0:
                layers.append(nn.Dropout(dropout))

            previous_dim = hidden_dim
```

Plot 2: Data + fitted line

You will see:

- noisy sine points
- the straight line learned by the neuron

Something like:

```
y
  *   *
  * *
-----/-----
 /
 /
----- x
```

The line will not follow the wave perfectly because one neuron is limited.

So in simple terms:

This section is teaching you that a neural network learns by repeatedly adjusting numbers (weights and biases) to reduce mistakes. A neuron is just a mathematical function, and gradient descent is the process that improves it.

After understanding this, the next step is to actually code the neuron manually.

📄 👍 💬 ↗️ ↺️ ⋮ | 📄 Convert to PDF

```

IndexError                                Traceback (most recent call last)
Cell In[21], line 127
    123     return history
    124
    125
    126
--> 127 tiny = run_experiment(hidden_sizes=(8,))
    128
    129 default = run_experiment(hidden_sizes=(64,32))
    130

Cell In[21], line 3
      1 def run_experiment(hidden_sizes=(64, 32), lr=1e-3, batch_size=32,
epochs=50, dropout=0.0):
      2
----> 3     model = FeedForwardNet(
      4         input_dim=X_train.shape[1],
      5         hidden_sizes=hidden_sizes,
      6         n_classes=7

```

Show less ^



The error is happening because your `FeedForwardNet` class assumes **two hidden layers**:

Python



Run

```

hidden_sizes[0]
hidden_sizes[1]

```



But in Experiment A you gave:

+ Ask anything



```
hidden_sizes=(8,)
```



```
FileNotFoundError                                Traceback (most recent call last)
Cell In[38], line 21
    17 n_classes=len(target_encoder.classes_)
    18 ).to(DEVICE)
    19
    20
--> 21 best_model.load_state_dict(torch.load("best_model.pth"))
    22 best_model.eval()
    23
    24
```

Show more ▾



Now the problem has changed. The `label_encoder` issue is fixed.

The **current error** is:

Python



Run

```
FileNotFoundError
best_model.load_state_dict(torch.load("best_model.pth"))
```

This means Python cannot find the file:

best_model.pth



in your current working folder.



#

- Error it was not created.

NameError: name 'test_loader' is not defined

5. Model loading with PyTorch 2.6+

- Sometimes:

Weights only load failed

because of newer `torch.load` behavior.

Send the traceback and I will tell you exactly what to change. Don't rerun random fixes yet.

       Convert to PDF

```
-----  
NameError                                Traceback (most recent call last)  
Cell In[25], line 16  
    12  
    13 best_model = FeedForwardNet(  
    14     input_dim=X_train.shape[1],  
    15     hidden_sizes=(64, 32),  
--> 16     n_classes=len(label_encoder.classes_)  
    17 ).to(DEVICE)  
    18  
    19 best_model.load_state_dict(torch.load("best_model.pth"))
```

Show more ▾



+ Ask anything



NameError: name 'label_encoder' is not defined



can u put everything together

Yes. Put everything in **one cell** (or split into a few cells if your lecturer prefers). This includes initialization, training, plotting loss, plotting the fit, and the **tanh removed comparison**.

```
<> Python Run

import numpy as np
import matplotlib.pyplot as plt

# =====
# Part 1.2: Hidden layers and activations
# 1 input -> 8 hidden neurons -> 1 output
# =====

# Initialise parameters
np.random.seed(42)

W1 = np.random.randn(1, 8) * 0.5
b1 = np.zeros(8)

W2 = np.random.randn(8, 1) * 0.5
b2 = np.zeros(1)

lr = 0.01
losses = []

# Training Loop
for step in range(3000):
```

+ Ask anything

Show more

The error is happening because your `FeedForwardNet` class assumes **two hidden layers**:

```
<> Python Run

hidden_sizes[0]
hidden_sizes[1]
```

But in Experiment A you gave:

```
<> Python Run

hidden_sizes=(8,)
```

That only has **one value**, so Python cannot find `hidden_sizes[1]`, causing the `IndexError`.

You need to update your `FeedForwardNet` class so it can handle different network sizes (tiny, default, large).

Replace your current `FeedForwardNet` class with this:

```
<> Python Run

class FeedForwardNet(nn.Module):
    def __init__(self, input_dim, hidden_sizes=(64, 32), n_classes=7):
        super().__init__()

        layers = []

        previous_dim = input_dim
```